Спринт 3 «Тест-кейси та звіти про дефекти»

Версія 1.0. Вересень 2026.

1. Паспорт спринта

Тижні: W6–7, два тижні: заняття 1 (Sprint Planning) + самостійна робота + заняття 2 (Demo, бліц, Retro). **Теми:** структура тест-кейсу; незалежні кейси; позитивні й негативні перевірки; класи еквівалентності та граничні значення; журнал виконання; дефект і звіт про дефект; правила зрозумілого звіту; severity і priority; життєвий цикл дефекту; системи відстеження дефектів; внутрішнє рецензування. **Вхід:** виправлена SRS і план тестування зі спринта 2; картка продукту з каталогу (demo, тестові облікові записи, обмеження). **Інкремент:** Досьє якості v3 — «Набір тест-кейсів + Журнал виконання + Реєстр дефектів». **Інструменти:** Google Sheets або Excel; GitHub Issues; demo-застосунок; Chrome або Firefox.

2. Замовлення і контекст

> *Замовлення: «Поточну версію продукту передано на тестування. Спроєктуйте тест-кейси за вимогами, виконайте їх і задокументуйте знайдені дефекти так, щоб розробник відтворив кожен без вашої участі».*

Demo-застосунок із картки каталогу є «поточною версією» вашого продукту, а виправлена у спринті 2 SRS — еталоном, з яким його порівнюють: усе, що demo робить не так, як записано у вимозі, — дефект. У спринті 4 частину кейсів ви автоматизуєте, у спринті 5 виконаєте повторно на «версії 2», тому кейси мають бути придатними для повторення іншими людьми, а звіти — для перевірки виправлення без вашої участі. Звіт, який неможливо відтворити за текстом, не зараховується.

3. Мета і результати навчання

Після спринта ви зможете:

- проєктувати незалежні тест-кейси зі структурою з шести полів, простежувані до вимог, з перевірюваними очікуваними результатами;
- добирати тестові дані за класами еквівалентності та граничними значеннями і пояснювати вибір;
- виконувати кейси на реальному застосунку і вести журнал зі статусами passed / failed / blocked / skipped та описом середовища;
- оформлювати звіт про дефект, за яким дефект відтворить людина, що не бачила ваших дій;
- обґрунтовано призначати severity і priority та пояснювати різницю між ними;
- вести реєстр дефектів у GitHub Issues і рецензувати кейси та звіти колег за чек-листами.

4. Теоретичний мінімум

Приклади — для продукту «ШопЛайн» (demo Toolshop, https://practicesoftwaretesting.com). Ідентифікатори вимог у прикладах умовні; у власних артефактах використовуйте ідентифікатори з виправленої SRS вашого продукту.

Частина А. Тест-кейси

4.1. Тест і тест-кейс

Тест (test) — набір операцій, призначених для отримання очікуваного результату в певній програмній системі; тест — це дія. **Тест-кейс (test case, тестовий випадок)** — зафіксований опис тесту, зазвичай рядок таблиці, де записано кроки разом з умовами виконання та очікуваними результатами з метою перевірити якість програми; тест-кейс — це документ. Один кейс виконують багато разів — на різних збірках, різними людьми; кожне виконання є окремим тестом із власним результатом у журналі. До вимоги прив'язують кейс, до кейсу — результати виконань і дефекти.

4.2. Структура тест-кейсу

Обов'язкове поле	Що містить	Приклад
1. Ідентифікатор	`TC-001`, `TC-002` …; унікальний, не змінюється; на нього посилаються журнал, звіти про дефекти, матриця простежуваності	`TC-003`
2. Пов'язана вимога	Ідентифікатор із виправленої SRS. Кейс без вимоги не простежується; вимога без кейсу не покрита	`F-60` «Додавання товару в кошик»

Обов'язкове поле	Що містить	Приклад
3. Заголовок	Одне речення: що перевіряється і за яких умов; читається без кроків	«Додавання товару в кошик із кількістю 2 без входу»
4. Кроки	Пронумерована послідовність; кожен крок — одна дія над одним елементом із конкретними даними	«3. У поле Quantity ввести 2»
5. Очікуваний результат	Для кожного кроку (або позначеної групи) — що має з'явитися чи змінитися, у перевірюваному вигляді: текст, адреса, число, стан елемента	«4. Лічильник кошика показує 2»
6. Передумови / умови виконання	Стан системи і дані до першого кроку: обліковий запис, роль, стан кошика чи рахунку, браузер, збірка	«Нова сесія (інкогніто), кошик порожній, без входу»

Рекомендовані поля: пріоритет кейсу (High / Medium / Low, успадковується від вимоги); статус (draft / reviewed / approved); тип (позитивний / негативний); автор; рецензент і дата; середовище; фактичний результат і статус виконання — два останні фіксуються у журналі, бо кейс один, а виконань багато.

4.3. Незалежні та залежні тест-кейси

Залежні кейси — ті, у яких виконання наступного залежить від попереднього: кроки продовжують один одного. **Незалежні** кейси не залежать один від одного: їх можуть виконувати різні люди у різних кімнатах, не радячись; незалежний кейс починається з дії «Відкрити застосунок (сторінку)» і завершується дією «Вийти з облікового запису / закрити застосунок». У спринті всі кейси незалежні: їх можна виконувати паралельно у будь-якому порядку, failed одного не «зносить» решту, кейс легко перетворюється на автотест (спринт 4) і повторюється на версії 2 (спринт 5). Ціна — повторювані кроки входу; їх виносять у передумови або виписують явно.

4.4. Покриття вимог. Позитивні й негативні кейси

Правило: **на кожну вимогу — щонайменше один кейс**, навіть з однієї дії. Обов'язково покриваються всі F- та GUI-вимоги; BL-вимоги — там, де правило видно через інтерфейс demo (решту покриють юніт-тести спринта 4); нефункціональні — ті, що перевіряються вручну. Для непокритих вимог у списку покриття зазначається причина.

Позитивний кейс перевіряє, що за дотримання правил система дає успішний результат; **негативний** — що за порушення правил вона відмовляє і зрозуміло повідомляє. Для «Входу покупця»: позитивний — коректні дані ведуть до кабінету; негативні — неправильний пароль дає повідомлення про помилку, порожній email не пропускається. Негативні кейси виявляють дефекти частіше; для кожної форми плануйте щонайменше одну негативну перевірку.

4.5. Класи еквівалентності та граничні значення

Клас еквівалентності (equivalence class) — множина вхідних значень, які система за вимогою обробляє однаково; достатньо перевірити одного представника. **Граничні значення (boundary values)** — значення на межах класів і поруч із ними, де найчастіші помилки «менше або дорівнює замість менше». Для діапазону [a..b] мінімальний набір: a−1, a, b, b+1; повний: a−1, a, a+1, b−1, b, b+1.

Поле і правило	Класи еквівалентності	Граничні значення
Рік народження: ціле від 1900 до поточного (2026)	Дійсний 1900..2026; недійсні: менше 1900; більше 2026; не ціле («19о5», 1985.5, порожньо)	1899, 1900, 1901, 2025, 2026, 2027
Кількість товару в рядку кошика: 1..99	Дійсний 1..99; недійсні: 0 і від'ємні; 100 і більше; дробові та нечислові	0, 1, 2, 98, 99, 100

Один кейс — один набір даних: якщо кейс із десятьма значеннями дав failed, незрозуміло, яке значення винне. Для серії значень — кілька коротких кейсів або один кейс із таблицею даних, де кожен рядок фіксується окремо. Біля значення зазначайте, чому воно обране («граничне: верхня межа 99»).

4.6. Приклад: три тест-кейси для «ШопЛайн»

ID	Вимога	Заголовок	Передумови	Кроки	Очікуваний результат	Пріоритет / тип
`TC-001`	`F-20`	Вхід покупця з коректними обліковими даними	Обліковий запис `customer@practicesoftwaretesting.com` / `welcome01` існує. Chrome, нова сесія (інкогніто)	1. Відкрити https://practicesoftwaretesting.com. 2. Натиснути «Sign in» у верхньому меню. 3. У поле Email ввести `customer@practicesoftwaretesting.com`, у поле Password — `welcome01`. 4. Натиснути кнопку Login. 5. У меню користувача натиснути «Sign out».	1. Головна сторінка з каталогом. 2. Сторінка входу з полями Email, Password і кнопкою Login. 3. Значення введено, повідомлень про помилку немає. 4. Сторінка кабінету (адреса містить `/account`); у меню замість «Sign in» — ім'я користувача. 5. У меню знову «Sign in».	High / позитивний
`TC-002`	`F-30`	Пошук товарів за частиною назви	У каталозі є товари зі словом «Hammer» у назві. Нова сесія, без входу	1. Відкрити https://practicesoftwaretesting.com. 2. У поле пошуку ввести `Hammer`. 3. Натиснути кнопку Search. 4. Переглянути список результатів до кінця. 5. Закрити вкладку.	1. Головна сторінка з каталогом. 2. У полі показано введений текст. 3. Список оновлено; над списком — ознака пошуку за «Hammer». 4. Кожен товар у списку містить «Hammer» у назві; інших немає.	High / позитивний
`TC-003`	`F-60`	Додавання товару в кошик із кількістю 2 без входу	Нова сесія (інкогніто), кошик порожній, без входу	1. Відкрити https://practicesoftwaretesting.com. 2. Натиснути на картку першого товару у списку. 3. У поле Quantity ввести `2`. 4. Натиснути «Add to cart». 5. Натиснути піктограму кошика у верхньому меню. 6. Закрити вкладку.	1. Головна сторінка з каталогом. 2. Сторінка товару з назвою, ціною, полем Quantity і кнопкою «Add to cart». 3. У полі Quantity — 2. 4. Повідомлення про додавання; лічильник кошика = 2. 5. У кошику один рядок: назва з кроку 2, кількість 2, сума = 2 × ціна з кроку 2.	High / позитивний

Назви елементів і адреси у прикладі наведено за станом demo на 8 вересня 2026 року; публічні demo оновлюються, тому перед виконанням звіряйте їх з екраном. Негативна пара до TC-003 — «Додавання товару з кількістю 0»: очікувано — товар не додано, біля

поля показано повідомлення про допустимий діапазон (формулювання — за GUI-вимогою вашої SRS), лічильник не змінився.

4.7. Журнал виконання

Кейс описує, що робити; журнал виконання (execution log) фіксує, що сталося під час конкретного прогону. Рядок журналу — одне виконання одного кейсу.

Статус	Коли ставиться	Що обов'язково додати
passed	Усі очікувані результати всіх кроків збіглися з фактичними	Дата, виконавець, середовище
failed	Хоча б на одному кроці фактичний результат відрізняється від очікуваного	Номер кроку, фактичний результат, Bug ID; failed без звіту не зараховується
blocked	Кейс неможливо довести до кроку, що перевіряється: заважає інший дефект (P1/P2), недоступне середовище, немає даних	Bug ID дефекту, що блокує, або опис проблеми середовища
skipped	Кейс свідомо не виконувався у цьому прогоні	Причина і хто ухвалив рішення

Стовпці: `Run` · `TC-ID` · `Дата` · `Виконавець` · `Збірка / середовище` (адреса demo і дата, браузер із версією, ОС) · `Статус` · `Bug ID` · `Коментар`. Середовище фіксуйте у кожному рядку: demo оновлюється і скидає дані, тож «на якій збірці» — перше питання розробника.

Частина Б. Звіти про дефекти

4.8. Дефект і звіт про дефект

Дефект (bug, defect) — розбіжність між фактичною поведінкою програми та вимогою. Еталон — виправлена SRS: якщо вимога каже «система показує повідомлення про помилку», а застосунок мовчить, — це дефект, навіть якщо нічого «не впало». Якщо вимога мовчить, а поведінка суперечить документації, логіці чи здоровому глузду, — теж дефект, але у звіті слід пояснити, на що ви спираєтеся. Поведінка, яка вам не подобається, але відповідає SRS, — не дефект, а пропозиція.

Звіт про дефект (bug report) — короткий і точний опис знайденого дефекту: єдиний канал від тестувальника до розробника, підстава для рішення «виправляти чи ні», частина історії якості. Якщо звіт недостатньо чіткий і зрозумілий, програміст не зможе виправити дефект: не відтворить, закриє як «не відтворюється», витратить час на листування, а дефект дійде до користувача.

4.9. Три вимоги до опису і момент створення

1. **Поясніть, як відтворити дефект** — розробники зазвичай ігнорують дефекти, які не можуть відтворити.
2. **Проаналізуйте дефект і опишіть його гранично стисло** — розлогий опис не читають.
3. **Складіть повний, зрозумілий і несуперечливий звіт** — звіт, що плутається, ускладнює виправлення.

Звіт складають **негайно після виявлення**: пізніше забуваються точні дані, порядок кроків, текст повідомлення. Одразу фіксуються кроки, фактичний результат і знімок екрана; решта — того ж дня.

4.10. Сім правил зрозумілого звіту

1. **Дефект описується словами; зображення лише ілюструє слова.** Кроки і результати зрозумілі без відкриття вкладень.
2. **Один дефект — один звіт.** Два дефекти в одному сценарії — два звіти; інакше один виправлять, другий загубиться.
3. **Уся інформація, потрібна для відтворення.** Уявіть, що звіт прочитають через пів року і відтворюватимуть на наступній версії: обліковий запис, дані, середовище, стан до першого кроку.
4. **Читач не знає, які дії ви виконували перед тим.** Кроки починаються з відкриття застосунку, а не з «після цього натиснути…».
5. **Краще один зрозумілий звіт, ніж кілька незрозумілих.** Незрозумілий повертається з питаннями або закривається.
6. **Грамотність; перечитати перед відправленням.** Друкарська помилка в назві кнопки змінює зміст кроку.
7. **Без сленгу.** Не «глючить», «вилітає», а «застосунок не відповідає», «сторінка перезавантажується».

4.11. Шаблон звіту про дефект

Поле	Зміст
Bug ID	`BUG-001` … (у GitHub Issues — номер issue)
Title	Коротке найменування: зрозуміле, унікальне, відображає суть і серйозність без читання опису; містить проблему та умову: «Зависання застосунку при збереженні файлу без розширення»
Severity, **Priority**	Серйозність впливу (4.12); пріоритет усунення (4.13)
Description	Чому це проблема (якщо неочевидно), вплив, умови виникнення; порушена вимога; обґрунтування severity і priority
Steps to reproduce	Найкоротша послідовність, яка завжди призводить до прояву дефекту; зрозуміла і розробнику, і іншому тестувальнику; кроки короткі й однозначні; за потреби — важливий проміжний результат
Actual result	Що отримано після кроків — прояв дефекту

Поле	Зміст
Expected result	Правильний результат після тих самих кроків — на основі специфікації (з ідентифікатором), документації, логіки, здорового глузду, досвіду
Environment	Адреса і дата збірки demo, браузер і версія, ОС, обліковий запис, роль
Attachments	Знімки екрана, відео, журнали; іменуються за Bug ID; кожне згадано у тексті
Linked test case / requirement	Ідентифікатори кейсу, на якому виявлено дефект, і порушеної вимоги
Службові	Reporter, Date, Status (4.14), Reviewer

4.12. Серйозність (severity)

Заповнює тестувальник, щоб показати вплив дефекту на функціональність і кінцевого користувача.

Рівень	Визначення	Приклад
Blocker	Блокує тестування	Demo не завантажується; неможливо увійти жодним тестовим обліковим записом
Critical	Відмова чи зависання системи; втрата даних; серйозний витік пам'яті; блокування ключової функціональності	Після оплати замовлення не створюється; переказ списує суму двічі («ФінКит»)
Major	Серйозна втрата функціональності: важливі функції працюють не повністю або не на належному рівні	Фільтр за ціною повертає товари поза діапазоном; заявку на відпустку не можна подати на наступний місяць («Персонал+»)
Normal	Користуватися можна, але є незручності	Лічильник кошика оновлюється лише після перезавантаження сторінки
Minor	Незначна втрата функціональності; проблема інтерфейсу чи зручності	Повідомлення про помилку не зникає після виправлення поля
Trivial	Косметичні дефекти	Орфографічна помилка у підписі кнопки

4.13. Пріоритет (priority)

Черговість усунення; використовується розробником для планування і зазвичай виставляється ним, але тестувальник пропонує значення й обґрунтовує його.

Рівень	Визначення	Наслідок для журналу
P1	Блокує будь-яке подальше виконання тест-кейсів	Усі невиконані кейси — blocked із цим Bug ID; викладача-«замовника» повідомляють негайно
P2	Блокує виконання поточного або кількох тест-кейсів	Заблоковані кейси перелічені у звіті й отримують blocked із цим Bug ID
P3	Усе інше	Кейси виконуються далі

Severity і priority незалежні: помилка в назві компанії на головній — Trivial, але P1 напередодні показу інвесторам; аварійне завершення у браузері, який продукт не підтримує, — Critical, але P3. У Description обґрунтовуйте обидва значення одним реченням кожне.

4.14. Життєвий цикл дефекту

New (звіт створено) → **Open** (підтверджено, призначено виконавця; альтернативи — Rejected: не дефект або дублікат, Deferred: відкладено, обидві з обґрунтуванням) → **Fixed** (виправлено, чекає перевірки в новій збірці) → **Verified** (кейс повторено на новій збірці, passed) або **Reopened** (failed; дефект повертається в Open) → **Closed**. У спринті 3 звіти перебувають у станах New і Open; перевірку виправлень ви виконаєте у спринті 5 на версії 2. У GitHub Issues станів два (open / closed), проміжні позначаються мітками `status:new` … `status:reopened`.

4.15. Системи відстеження дефектів. GitHub Issues

Система відстеження дефектів (bug tracking system) дає єдиний реєстр з унікальними ідентифікаторами, станами, історією, призначенням виконавців, зв'язками з кейсами й вимогами, пошуком дублікатів і метриками (Jira, GitHub Issues, GitLab Issues, YouTrack, Azure DevOps, Redmine). Рекомендовано **GitHub Issues** у репозиторії команди: доступ мають усі, у спринті 4 туди ж піде код автотестів, а посилання на issue у коміті зв'яже звіт із тестом. Налаштування у перший день спринта: шаблон issue `.github/ISSUE_TEMPLATE/bug_report.md` за формою нижче; мітки `severity:blocker` … `severity:trivial`, `priority:P1` … `priority:P3`, `status:*`, `sprint:3`; кожен звіт — окрема issue з усіма трьома групами міток; зв'язок із кейсом і вимогою — у тексті звіту, а у таблиці кейсів — стовпець із номером issue. Допускається таблиця `bug-reports.md` з тими самими полями.

```
---
name: Звіт про дефект
labels: ["status:new", "sprint:3"]
---
**Severity:** Blocker | Critical | Major | Normal | Minor | Trivial   **Priority:** P1 | P2 | P3
**Linked:** test case TC-___; requirement ___
### Description
### Steps to reproduce
1. Відкрити <адреса> у <браузер> (нова сесія).
### Actual result
### Expected result
### Environment   demo <адреса, дата>; <браузер, версія>; <ОС>; обліковий запис
### Attachments   BUG-___-1.png — що на знімку
```

4.16. Два приклади звіту: хороший і поганий

Приклад 1 — хороший (дефект умовний, для ілюстрації).

```
Bug ID:    BUG-007
Title:     Кошик: кількість 100 приймається, хоча допустимий діапазон 1..99 (F-60)
Severity:  Major        Priority: P3        Linked: TC-004; F-60, GUI-40
Description:
  F-60 (виправлена SRS v1.1) обмежує кількість одиниць у рядку кошика діапазоном 1..99,
  GUI-40 вимагає повідомлення при виході за межі. Фактично сторінка товару приймає 100
  і додає товар; підсумок обчислено для 100 одиниць. Значення 1, 2, 98, 99 працюють
  правильно; 0 та -1 — окремі звіти BUG-008, BUG-009. Severity Major: контроль кількості
  не працює, але замовлення з допустимою кількістю оформити можна. P3: кейсів не блокує.
Steps to reproduce:
  1. Відкрити https://practicesoftwaretesting.com у Chrome, нова сесія (інкогніто).
  2. Натиснути на картку товару «Combination Pliers» на головній сторінці.
  3. У поле Quantity ввести 100.
  4. Натиснути кнопку «Add to cart».
  5. Натиснути піктограму кошика у верхньому меню.
Actual result:
  Після кроку 4 показано повідомлення про додавання; лічильник кошика = 100.
  Після кроку 5 у кошику рядок «Combination Pliers», Quantity 100, сума = 100 × ціна.
Expected result:
  Після кроку 4 товар не додано; біля поля Quantity — повідомлення про допустимий
  діапазон 1..99; лічильник = 0. Після кроку 5 кошик порожній.
Environment: demo 2026-10-06; Chrome 129.0; macOS 15; без входу в обліковий запис.
Attachments: BUG-007-1.png — сторінка товару після кроку 4; BUG-007-2.png — кошик після кроку 5.
```

Чому добрий: заголовок містить проблему й умову; є вимога-еталон; кроки починаються з відкриття сторінки, кожен — одна дія з конкретними даними; результати прив'язані до кроків і перевірювані; severity і priority обґрунтовані; суміжні дефекти — окремі звіти; середовище зазначене; вкладення пояснені словами.

Приклад 2 — поганий.

```
Bug ID: BUG-011      Title: Кошик глючить!!!      Severity: Blocker   Priority: P1
Description: Додавав товари в кошик і все поламалось, сума неправильна. Ще й на сторінці
  товару фото криво стоїть. Див. скрін.
Steps to reproduce: 1. Зайти на сайт. 2. Покласти щось у кошик. 3. Подивитись.
Actual result: Не працює.      Expected result: Має працювати.
Environment: —                 Attachments: Screenshot 2026-10-06 at 14.03.11.png
```

Розбір за правилами: (1) суть — «див. скрін», текст не каже, що саме неправильно у сумі; (2) два дефекти в одному звіті — сума і зображення; (3) невідомі товар, кількість, очікувана й отримана суми, середовище; (4) «покласти щось у кошик» читач не повторить, чи був вхід — невідомо; (5) звіт написано за хвилину, розробник витратить годину і закриє як «не відтворюється»; (6) знаки оклику в заголовку, «має працювати» замість результату за вимогою; (7) «глючить», «поламалось», «скрін». Blocker і P1 без обґрунтування: якщо інші кейси виконуються, це не Blocker і не P1.

5. Завдання спринта

5.1. Спільний обсяг робіт

1. Скласти перелік вимог виправленої SRS до покриття і розподілити їх між членами команди; узгодити, які BL- та нефункціональні вимоги не покриваються у спринті 3 і чому.
2. Підготувати середовище (адреса demo, облікові записи, браузер) і трекер: GitHub Issues із шаблоном і мітками за 4.15 або `bug-reports.md`.
3. Спроєктувати тест-кейси за нормативом (розділ 6): шість обов'язкових полів, незалежні, позитивні й негативні, з даними за класами еквівалентності та граничними значеннями щонайменше для двох вимог із діапазонами; таблиця `test-cases.xlsx`.

4. Рецензувати кожен кейс за чек-листом 5.3 з позначкою рецензента; виправити зауваження.
5. Виконати всі кейси на demo; вести `execution-log.xlsx` (статус, дата, виконавець, середовище, Bug ID).
6. Для кожного failed скласти звіт за шаблоном 4.11 у трекері; разом — за нормативом, щонайменше три різні рівні severity, severity і priority обґрунтовані.
7. Рецензувати кожен звіт за семи правилами; для 5 осіб — Аудитор-опонент відтворює три дефекти лише за текстом звітів.
8. Звести інкремент: список покриття «вимога → кейси», статистика (кейсів усього / passed / failed / blocked; дефектів за severity), `README.md`; таблиця розподілу задач; протокол ретро; демо.

Дефектів на demo достатньо, якщо шукати там, де вони живуть: негативні кейси, граничні значення, форми з вкладеннями, повторні дії (двічі натиснути кнопку), різні браузери, розбіжності між текстами інтерфейсу та SRS.

5.2. Розбивка за ролями

Роль	Задачі	Артефакти
Керівник тестування	Декомпозиція замовлення; розподіл вимог між членами команди; контроль покриття і нормативу; рішення щодо P1/P2 і blocked; зведення інкременту і статистики; представлення демо	`README.md` зі списком покриття і статистикою; таблиця розподілу задач
Тест-аналітик	Проєктування кейсів (структура, незалежність, позитивні/негативні, добір даних); простежуваність кожного кейсу до вимоги; список непокритих вимог із причинами	`test-cases.xlsx` (аркуші «Test cases», «Coverage»)
Інженер з тестування	Середовище й облікові записи; налаштування трекера; виконання кейсів; журнал; звіти про дефекти з вкладеннями; зв'язок звітів із кейсами	`execution-log.xlsx`; issues (або `bug-reports.md`); `attachments/`
Скрам-майстер / рецензент	Таблиця задач і standup; рецензування КОЖНОГО кейсу і КОЖНОГО звіту за чек-листами 5.3 з позначкою і датою; повернення на доопрацювання; протокол ретро; підготовка до бліцу	Аркуш «Review» у `test-cases.xlsx`; позначки Reviewer у звітах; протокол ретро
Аудитор-опонент (5 осіб)	До демо, не бачивши виконання, обирає три звіти різних severity і намагається відтворити дефекти лише за текстом; фіксує «відтворено / не відтворено» і що заважало; перевіряє інкремент на DoD; формулює очікувані питання замовника	`audit.md`: «Bug ID — відтворено (так/ні) — час — зауваження»; перелік питань до демо

Команда з 3 осіб: Скрам-майстер/рецензент поєднується з Керівником; рецензування не скасовується, але рецензентом кейсу чи звіту не може бути його автор.

5.3. Чек-листи рецензента

Чек-лист тест-кейсу (усі 8 пунктів «так» — кейс отримує статус reviewed і позначку рецензента):

1. Заповнено всі шість обов'язкових полів; ідентифікатор унікальний за схемою `TC-NNN`.
2. Вказано ідентифікатор вимоги з виправленої SRS, і кейс перевіряє саме цю вимогу.
3. Кейс незалежний: починається з відкриття застосунку або сторінки, завершується виходом чи закриттям, не спирається на стан, створений іншим кейсом.
4. Передумови повні: обліковий запис, дані, стан кошика/рахунку/профілю, браузер.
5. Кожен крок — одна конкретна дія з конкретними даними (не «ввести коректні дані», а «ввести `welcome01`»).
6. Кожен крок або позначена група має очікуваний результат у перевірюваному вигляді (текст, адреса, число, стан), без «працює коректно».
7. Заголовок відображає суть і умову, читається без кроків; тип (позитивний/негативний) зазначено.
8. Тестові дані обґрунтовані: для діапазонів — граничні значення; для форм — є негативна пара.

Чек-лист звіту про дефект — сім правил 4.10 у формі питань: (1) чи зрозумілий дефект без відкриття вкладень; (2) чи описано рівно один дефект; (3) чи вистачить інформації, щоб відтворити дефект через пів року на іншій версії; (4) чи починаються кроки з відкриття застосунку і чи не припускають прихованих дій; (5) чи не залишилося питань, які рецензент мусив би поставити автору; (6) чи немає помилок у назвах елементів і даних; (7) чи немає сленгу. Додатково до DoD: severity і priority обґрунтовані в Description; є Linked test case / requirement; Expected result посилається на вимогу. Рецензент ставить позначку і дату у полі Reviewer; звіт із зауваженнями повертається автору.

5.4. Етика роботи з публічним demo

Demo-застосунки — публічні ресурси, якими одночасно користуються інші команди і люди по всьому світу. Заборонено: навантажувальні перевірки і масові автоматичні запити; зміна паролів спільних тестових облікових записів; видалення чи псування чужих даних в адмінпанелях; введення реальних персональних даних (імена, телефони, адреси, номери карток); вставка скриптів чи атакувальних рядків у публічні поля (контактні форми, коментарі); образливі або рекламні тексти. Використовуйте вигадані дані і власні облікові записи там, де реєстрація дозволена. Вимоги до захищеності перевіряються лише спостереженням (маскування пароля, поведінка після виходу), без спроб обійти захист. Порушення етики — 0 за командну частину спринта.

6. Нормативи за розміром команди

Норматив	3 особи	4 особи	5 осіб
Тест-кейсів (спринт 3)	≥15	≥20	≥25
Звітів про дефекти demo-застосунку (спринт 3)	≥6	≥8	≥10
Рецензовано кейсів і звітів; рівнів severity серед звітів	100 %; щонайменше три різні рівні — незалежно від розміру		
Відтворення дефектів Аудитором-опонентом за текстом звіту	—	—	3 звіти

7. Definition of Done

- Тест-кейсів не менше нормативу; у кожному заповнено всі шість обов'язкових полів; кожен простежується до вимоги виправленої SRS; усі F- та GUI-вимоги покриті або внесені у список непокритих із причиною.
- Усі кейси виконані на demo; для кожного у журналі — статус, дата, виконавець, середовище; для failed — Bug ID, для blocked — причина.
- Звітів не менше нормативу; щонайменше три різні рівні severity; для кожного failed-кейсу є звіт; кожен звіт містить усі поля шаблону 4.11.
- Кожен звіт і кожен кейс пройшли рецензування — є позначка рецензента з датою; зауваження виправлені.
- Трекер налаштовано, посилання є в README; для 5 осіб — `audit.md` заповнений.
- Таблиця розподілу задач і протокол ретро здані.

8. Структура інкременту

```
docs/sprint3/
  README.md            зміст досьє v3; посилання на трекер (issues з міткою sprint:3); статистика (кейсів
                       усього/passed/failed/blocked; дефектів за severity і priority); список покриття
                       «вимога → TC-…» або «не покрито: причина»
  test-cases.xlsx      «Test cases»: ID, Вимога, Заголовок, Передумови, Кроки, Очікуваний результат,
                       Пріоритет, Тип, Автор, Рецензент, Дата рецензування, Статус кейсу
                       «Coverage»: ID вимоги, Формулювання (скорочено), TC-…, Примітка
                       «Review»: TC-ID, Рецензент, Дата, Зауваження, Виправлено (так/ні)
  execution-log.xlsx   «Run 1»: Run, TC-ID, Дата, Виконавець, Середовище, Статус, Bug ID, Коментар
                       «Environment»: адреса demo, дата, браузери і версії, ОС, облікові записи
  bug-reports.md       лише якщо трекер не використовується: звіти за шаблоном 4.11 з позначкою Reviewer
  attachments/         BUG-007-1.png …  (іменування за Bug ID)
  audit.md             для 5 осіб: результати відтворення трьох дефектів, питання до демо
  task-board.md        таблиця розподілу задач;   retro.md — протокол Start/Stop/Continue
```

Google Sheets допускається як робочий інструмент; наприкінці спринта таблиці експортуються у `.xlsx` у репозиторій.

9. Хід спринта

Заняття 1 (W6) — Sprint Planning, ~30 хв на команду, паралельно. 0–5 хв: уточнення замовлення з викладачем-«замовником», підтвердження ролей після ротації. 5–15 хв: перелік вимог до покриття, розподіл між членами, погодження непокритих. 15–25 хв: декомпозиція в задачі, таблиця розподілу, схема ідентифікаторів, трекер. 25–30 хв: перевірка доступу до demo і облікових записів, дата стану demo для журналу. Решта заняття — налаштування трекера і перші 3–5 кейсів разом, щоб узгодити стиль кроків та очікуваних результатів.

Міжзаняттєвий період. W6, дні 1–3: усі кейси у статусі draft; standup у чаті (що зроблено / що блокує). W6, дні 4–5: рецензування кейсів, виправлення, статус reviewed. W7, дні 1–3: виконання, журнал, звіти про дефекти негайно після виявлення. W7, дні 4–5: рецензування звітів, робота Аудитора-опонента, README, репетиція демо (10 хв із секундоміром).

Заняття 2 (W7) — Demo + бліц + Retro. Демо 10 хв на команду: 2 хв — покриття і статистика; 4 хв — виконання наживо двох кейсів (позитивний і негативний) із журналом; 3 хв — два звіти з трекера різних severity з обґрунтуванням; 1 хв — що не вдалося і чому. Бліц із розділу 11 — довільному члену команди на вибір викладача (для 5 осіб — двом). Після демо всіх команд — ретроспектива 15 хв за шаблоном Start/Stop/Continue, протокол здається; анонімна peer-review форма — 5 хв. **До здачі:** `docs/sprint3/` за розділом 8, посилання на трекер, таблиця розподілу, протокол ретро.

10. Оцінювання спринта

Складова	Бали	Критерії для інкременту спринта 3
Повнота (команда)	2	Кейсів і звітів не менше нормативу; усі F- та GUI-вимоги покриті або обґрунтовано непокриті; всі кейси виконані; ≥3 рівні severity; всі поля шаблонів заповнені

Складова	Бали	Критерії для інкременту спринта 3
Коректність (команда)	2	Кейси незалежні; очікувані результати перевірювані і прив'язані до кроків; дані для діапазонів — граничні; severity і priority відповідають визначенням 4.12–4.13 і обґрунтовані; статуси журналу — за 4.7
Обґрунтованість і документування (команда)	2	Кожен failed має звіт; кожен звіт відтворюваний за текстом (викладач відтворює вибірково); є позначки рецензентів; README зі статистикою і покриттям; структура за розділом 8
Виконання ролі (індивідуально)	3	За артефактами ролі (5.2), таблицею розподілу і позначками авторства/рецензування
Бліц (індивідуально)	1	Коротка правильна відповідь на питання з розділу 11 або пояснення власного артефакту

Коригування — за концепцією: peer-review коефіцієнт 0,8–1,1 за систематичних сигналів; штраф −1 за незданий протокол ретро або таблицю розподілу; доздача протягом тижня зі штрафом −20 % командної частини; бонус +0,5 за змістовне питання чи знайдену помилку на демо іншої команди.

11. Бліц-питання спринта

1. Чим тест-кейс відрізняється від тесту?
2. Назвіть шість обов'язкових полів тест-кейсу.
3. За якою ознакою визначити, що тест-кейс незалежний?
4. Чому одна вимога зазвичай потребує більше ніж одного кейсу?
5. Поле приймає значення від 1 до 99. Які граничні значення ви перевірите і чому?
6. Чим відрізняються статуси failed і blocked? Що обов'язково додати до кожного?
7. Що таке дефект? Чи є дефектом поведінка, яка відповідає SRS, але вам не подобається?
8. Назвіть три вимоги до опису дефекту.
9. Чому «один дефект — один звіт»? Що станеться інакше?
10. Чим Severity відрізняється від Priority? Наведіть приклад Trivial із P1.
11. Що означає priority P2 і як він відображається у журналі виконання?
12. Опишіть життєвий цикл дефекту від New до Closed. Коли дефект стає Reopened?

12. Типові помилки й поради

- **Залежні кейси.** «TC-007: продовжити з кроку 5 TC-006» — такий кейс не виконати окремо і не автоматизувати. Кожен кейс починається з відкриття сторінки.
- **Неперевірювані очікувані результати.** «Система працює коректно» — рецензент не визначить passed чи failed. Пишіть текст, адресу, число.
- **Кейс без даних.** «Ввести коректний email» змушує виконавця вигадувати дані і робить результат невідтворюваним.
- **Failed без звіту або один звіт на кілька failed.** Кожен failed має свій Bug ID; кожен звіт — один дефект.
- **Severity «на око».** Усі дефекти Major або всі Critical — ознака, що визначення 4.12 не застосовували. Обґрунтуйте рівень одним реченням.
- **Звіт без середовища й дати.** Demo скидає дані й оновлюється; без дати збірки звіт неможливо перевірити у спринті 5.
- **Рецензування «заднім числом».** Позначки без зауважень на 20 кейсах за годину — сигнал для викладача; рецензуйте у міру створення.
- **Демо без живого виконання.** Показуйте кейс і дефект наживо на demo, а не лише таблиці: це і є доказ відтворюваності.